

Informe del módulo de adquisición de datos

CSDI RAG Engine

1 Módulo de adquisición de datos

El módulo de adquisición de datos constituye la etapa inicial del sistema de recuperación de información. Su responsabilidad es obtener documentos desde fuentes previamente definidas, extraer de ellos contenido textual útil y entregar una representación estructurada que pueda ser procesada posteriormente por los módulos de persistencia, segmentación e indexación. En esta fase no se realiza una búsqueda abierta sobre Internet; el proceso trabaja sobre fuentes controladas, configuradas de forma explícita según el dominio temático del sistema.

Diseño general del flujo de adquisición. La adquisición se implementó como un pipeline desacoplado compuesto por cinco etapas: *configuración de la fuente*, *crawling*, *scraping*, *persistencia del documento fuente* y *preparación de fragmentos indexables*. Esta separación permite que cada responsabilidad tenga límites técnicos claros: el crawler descubre y descarga páginas, el scraper interpreta el HTML y extrae información relevante, y el orquestador conecta esos resultados con el resto del flujo de ingesta.

El punto de entrada de este flujo es **IngestionOrchestrator**. Este componente recibe un identificador de fuente, recupera su configuración, ejecuta el crawler, procesa cada página descargada con el scraper y transforma cada documento extraído en unidades más pequeñas de texto. La decisión de centralizar la coordinación en un orquestador evita que el crawler o el scraper conozcan detalles de almacenamiento o de indexación, lo cual reduce el acoplamiento entre componentes y facilita validar cada etapa de forma independiente.

Configuración explícita de fuentes. Cada fuente se describe mediante una configuración declarativa. Esta configuración incluye las URLs semilla, dominios permitidos, prefijos de ruta aceptados, patrones de ruta bloqueados, profundidad máxima de exploración, límite de páginas, extensiones excluidas, tipos de contenido admitidos, política de respeto a `robots.txt`, tiempo de espera por petición, número de reintentos y selectores CSS usados por el scraper.

Esta decisión es importante porque el módulo de adquisición no debe descargar contenido indiscriminadamente. Al imponer fronteras por dominio y por ruta, el sistema mantiene el corpus alineado con el dominio seleccionado y evita introducir páginas irrelevantes, duplicadas o administrativas. Además, los selectores de scraping se declaran por fuente, por lo que el mismo código puede procesar sitios con estructuras HTML diferentes sin incorporar reglas rígidas dentro de la lógica del scraper.

Crawler: descubrimiento controlado de documentos. El crawler está implementado como una exploración en anchura (*Breadth-First Search*). Parte de las URLs semilla definidas para una fuente y va descubriendo enlaces internos hasta alcanzar la profundidad máxima configurada o el límite total de páginas. La elección de BFS es adecuada para documentación técnica porque prioriza primero las páginas cercanas a los puntos de entrada principales, que suelen corresponder con guías,

índices de referencia y secciones de alto valor informativo. De esta forma, el corpus inicial se construye desde las páginas más representativas antes de descender hacia contenido más específico.

Antes de descargar una URL, el crawler aplica varias validaciones. Primero normaliza la dirección eliminando fragmentos y, salvo configuración contraria, cadenas de consulta. Esto evita procesar como documentos distintos páginas que solo cambian por anclas internas o parámetros no relevantes. Luego verifica que el esquema sea HTTP o HTTPS, que el dominio pertenezca a la lista permitida, que la ruta coincida con los prefijos aceptados, que no contenga patrones bloqueados y que no termine en extensiones descartadas como imágenes, videos, archivos comprimidos u otros formatos que no forman parte de esta etapa textual.

El crawler también valida el tipo de contenido devuelto por el servidor. Por defecto, solo acepta `text/html`, lo cual es coherente con un módulo diseñado para extraer texto estructurado desde páginas de documentación. Esta restricción reduce errores posteriores en el scraper y evita intentar interpretar como HTML recursos binarios o archivos auxiliares.

Política de acceso y robustez del crawler. La implementación incorpora respeto a `robots.txt`. Para cada dominio se consulta y cachea la política correspondiente mediante `RobotFileParser`. Si el sitio define un `crawl-delay`, el crawler lo combina con el retardo configurado localmente y utiliza el mayor valor. Esta estrategia justifica técnicamente el comportamiento del módulo: la adquisición de datos no solo debe ser funcional, también debe evitar sobrecargar las fuentes y seguir las reglas publicadas por los administradores del sitio.

El crawler maneja fallos de red mediante reintentos configurables. Las respuestas con errores de servidor pueden reintentarse, mientras que las respuestas no exitosas definitivas se descartan sin detener todo el proceso. Esta política permite que una página defectuosa no interrumpa la adquisición completa de la fuente. Además, el uso de un `User-Agent` propio identifica el sistema de forma explícita durante las peticiones HTTP.

Scraper: extracción estructurada del contenido. El scraper recibe el HTML descargado por el crawler y lo transforma en un documento estructurado. Para ello utiliza `BeautifulSoup` con el parser `lxml`, combinación apropiada para recorrer árboles HTML de forma tolerante ante pequeñas inconsistencias del marcado. El resultado del scraping contiene la URL, el título, el contenido principal, la ruta de navegación (*breadcrumb*), los bloques de código y el identificador de la fuente.

La extracción no se basa en tomar todo el texto visible de la página. Antes de seleccionar el contenido, el scraper elimina nodos irrelevantes definidos en la configuración, como `script`, `style`, `noscript`, encabezados, pies de página, barras laterales, menús de navegación y otros elementos que suelen introducir ruido. Esta limpieza es esencial en un sistema de recuperación: si se indexan menús, enlaces repetidos o texto de interfaz, el recuperador puede asignar relevancia a fragmentos que no responden realmente la consulta del usuario.

Selectores configurables y tolerancia a estructuras distintas. El scraper utiliza listas ordenadas de selectores CSS para localizar el título, el cuerpo principal, las migas de navegación y los bloques de código. Para el título y el contenido principal se toma el primer selector que produzca una coincidencia válida. Esta política permite definir selectores más específicos al inicio y dejar selectores más generales como respaldo. Por ejemplo, una fuente puede priorizar `main[role="main"]` y luego caer a selectores como `.body` o `.document` si la estructura de la página cambia entre secciones.

Esta decisión ofrece un equilibrio entre precisión y robustez. Un selector único sería frágil ante variaciones del HTML; por otro lado, extraer todos los nodos posibles aumentaría el ruido. La lista ordenada de selectores permite ajustar cada fuente sin modificar código, manteniendo el scraper genérico y reutilizable.

Tratamiento de títulos, contenido y bloques de código. El título se considera obligatorio porque aporta una señal semántica fuerte para identificar el documento y contextualizar sus fragmentos. Si no se encuentra título o contenido principal, la página se descarta. Esta decisión evita almacenar documentos incompletos que luego podrían afectar la calidad de la indexación.

El contenido principal se extrae como texto plano usando separadores de espacio, de modo que la estructura visual del HTML no introduzca saltos o concatenaciones incorrectas. Los bloques de código se extraen por separado y se deduplican durante el procesamiento. Esta separación resulta especialmente importante en el dominio de documentación técnica, donde los ejemplos de código son información relevante pero tienen una estructura distinta al texto explicativo. Mantenerlos identificados permite conservar esa señal para etapas posteriores sin mezclarla de forma descontrolada con menús, enlaces o elementos decorativos.

Integración con la preparación para indexación. Una vez obtenido un documento estructurado, el orquestador lo persiste como documento fuente y lo envía al proceso de segmentación. La segmentación divide el contenido en fragmentos con tamaño objetivo y solapamiento, preservando metadatos como fuente, URL, título y breadcrumb. La adquisición no termina en descargar HTML: su salida debe ser texto limpio, trazable y apto para ser indexado por los modelos de recuperación.

Esta trazabilidad es una decisión técnica central. Cada fragmento conserva su procedencia, por lo que un resultado recuperado puede relacionarse con el documento original y con la fuente desde la que fue adquirido. Esto permite presentar evidencias al usuario, depurar errores de recuperación y reconstruir el origen de la información almacenada.

Justificación técnica de la separación crawler–scraper. Separar crawler y scraper evita mezclar dos problemas diferentes. El crawler resuelve navegación, políticas de acceso, normalización de URLs, límites de profundidad, filtrado y descarga. El scraper resuelve interpretación del documento, limpieza de HTML y extracción semántica. Esta división hace que el sistema sea más mantenible: cambiar una regla de crawling no afecta la lógica de extracción, y ajustar los selectores de una fuente no modifica la forma en que se descubren enlaces.

La implementación también favorece la extensibilidad. Nuevas fuentes pueden incorporarse mediante configuración, siempre que expongan contenido HTML procesable. En ese caso se definen sus semillas, límites y selectores, mientras que el pipeline conserva el mismo flujo de ejecución. Así, el módulo cumple su función dentro de la arquitectura general: construir un corpus inicial controlado, limpio y técnicamente justificable para alimentar los componentes posteriores del sistema de recuperación de información.